Companion to Bird & de Moor, Algebra of Programming

§5. Datatypes in Allegories

§5.1. Relators

(5.1a)

A **relator** `F : 𝒞↛𝒟` is a 2-functor between allegories: it preserves `𝟙`, composition and `⊑`.

(5.1b)

	the statement
1	`Map(f) ⇒ Map(F(f))` and `Map(f) ⇒ F(f°) = F(f)°` . Lemma 5.1
2	Over a **tabular** allegory a functor is a relator `⇔` it preserves `°`. Theorem 5.1 the book's reverse direction (preserving `°` ⟹ relator) needs the source to be `Rel`: the counterexample over a tabular allegory that is not `Rel` shows it can fail
3	`F(R°) = F(R)°` for every `R`, so `F(R)°` needs no bracket.
4	Two relators agreeing on maps are equal. Corollary 5.1
5	`Coreflexive(X); Coreflexive(Y) ⇒ F(X ∩ Y) = F(X) ∩ F(Y)` Ex 5.2
6	`F(R ∩ S) ⊑ F(R) ∩ F(S)` , and strictly. Ex 5.2
7	`F(dom(R)) = dom(F(R))` Ex 5.5

The **power relator** `P` — `xs P(R) ys⇔(∀a∈xs. ∃b∈ys. a R b)∧(∀b∈ys. ∃a∈xs. a R b)` — is where the fourth is strict: for `R={(a₁,b₁),(a₂,b₂)}` and `S={(a₁,b₂),(a₂,b₁)}` the pair `({a₁,a₂},{b₁,b₂})` is in `P(R)∩P(S)`, while `R∩S=∅`.

§5.2. Relational products

(5.1)

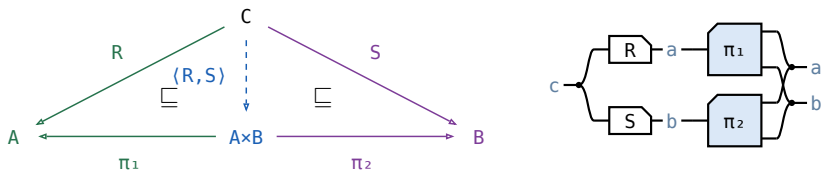
⟨R,S⟩ ≜ (R π₁°) ∩ (S π₂°) — the **fork** of R : C→A and S : C→B, with (π₁,π₂) the tabulation of ⊤.

⟨R,S⟩≜Rπ₁°∩Sπ₂° c ⟨R,S⟩ (a,b)⇔c R a∧c S b.

(5.2a)

⟨R,S⟩ π₁ = dom(S) R (5.6) ⟨R,S⟩ π₂ = dom(R) S (5.7)

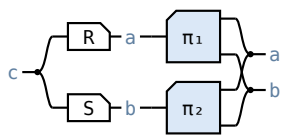
(5.2b)



A domain is coreflexive, so ⟨R,S⟩π₁⊑R, with equality exactly when S is entire; for maps both triangles commute and ⟨f,g⟩ is unique. In Rel, c ⟨R,S⟩ (a,b) iff c R a and c S b — copy c, then R on one strand and S on the other, which is ◁(R⊗S) on the right.

No ° survives the translation. π₁=𝟙⊗⊸ discards the second component, so π₁°=𝟙⊗⊸ **creates** one out of nothing, and ∩ is copy, run both, merge. Draw that and the created strands — the two dots with no left end, and the crossing they force — are merged against real ones, which is the monoid's unit law:

(5.2c)



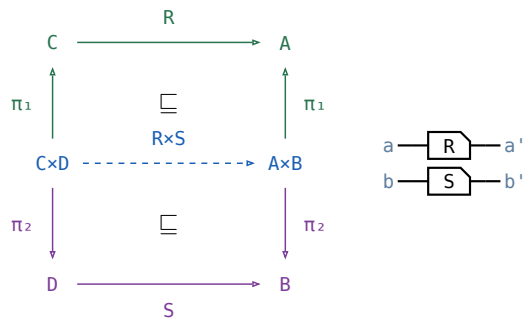
⊸▷=𝟙 on each half

§5.2.1. Relational product R×S

(5.2)

R × S ≜ ⟨π₁ R,π₂ S⟩ — the relational product R×S≜⟨π₁R,π₂S⟩.

(5.2.1a)

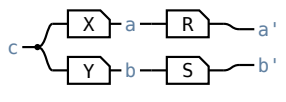


Right-then-up is (R×S)π₁, up-then-right is π₁R, and (R×S)π₁⊑π₁R, equality when S is entire. In Rel, (c,d) (R×S) (a,b) iff c R a and d S b — two strands side by side, no copy dot: R×S=R⊗S.

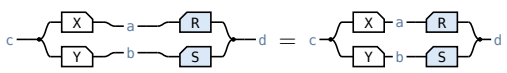
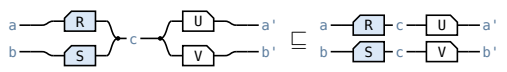
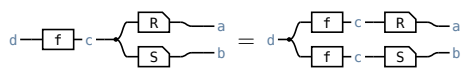
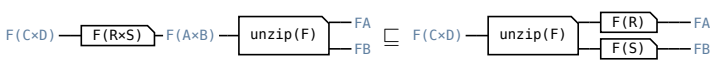
§5.2.2. Absorption

For X : E→C and Y : E→D, ⟨X,Y⟩(R×S)=⟨XR,YS⟩. Both sides are this picture:

(5.3)

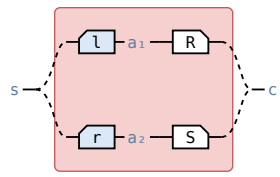
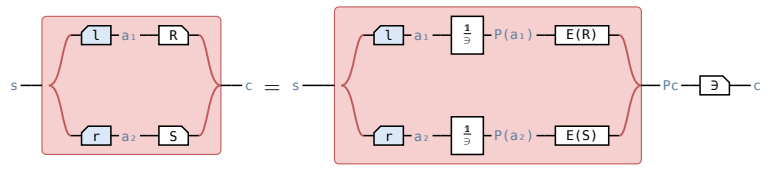
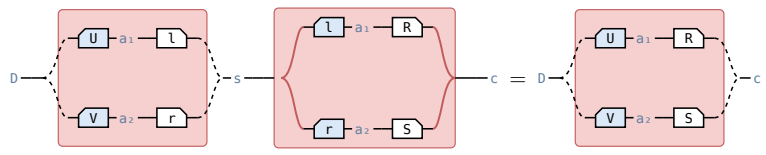


(5.2.2a)

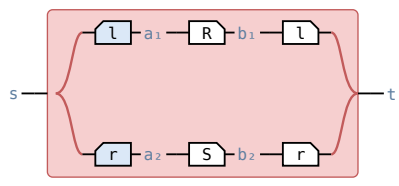
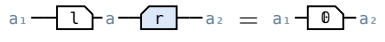
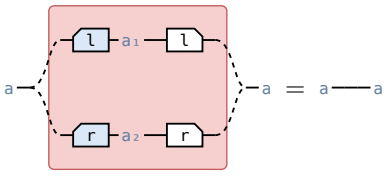
	the law	picture
1	`⟨X,Y⟩ ⟨R,S⟩° = (X R°) ∩ (Y S°)`	
2	`⟨R,S⟩° ⟨U,V⟩ ⊑ (R° U) × (S° V)`	
3	`Map(f) ⇒ f ⟨R,S⟩ = ⟨f R,f S⟩` `f` a map; it fails for an arbitrary arrow;	
4	`X ⟨R,S⟩ ⊑ ⟨X R,X S⟩` the half of the row above that holds for every `X`	
5	`F(R × S) unzip(F) ⊑ unzip(F) (F(R) × F(S))` `unzip(F)≜⟨F(π₁),F(π₂)⟩`; only `⊑` for an arbitrary `R`, `S`	
6	`g=curry(f)⇔(g×𝟙)eval=f` reading `×` as the relational product, does `Rel` have exponentials?	

§5.3. Relational coproducts

(5.3a)

	the statement R : $a_1 \to c$, S : $a_2 \to c$	picture
1	[R,S] ≜ (l° R) ∪ (r° S) (5.9) The tape is the union — a particle entering at A+B takes exactly one branch — and the two mirrored boxes are what makes the branches disjoint.	
2	[R,S] = [$\frac{R}{∋}$,$\frac{S}{∋}$] ∋	
3	l [R,S] = R , r [R,S] = S l T = R; r T = S ⇛ T = [R,S] ,	
4	U : $a_1 \to D$, V : $a_2 \to D$ [U,V]° [R,S] = (U° R) ∪ (V° S) (5.11)	
5	Z : c→D [R,S] Z = [R Z,S Z]	

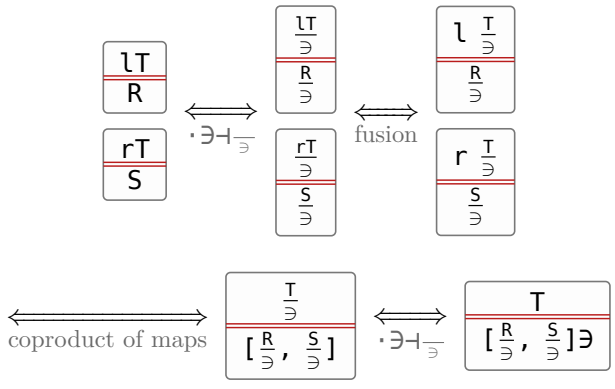
(5.3b)

	the statement R : $a_1 \to b_1$, S : $a_2 \to b_2$	picture
1	R + S ≜ [R l,S r] (5.10)	
2	l l° = 𝟙 , r r° = 𝟙 Ex 5.12	
3	l r° = 𝟘 , r l° = 𝟘 Ex 5.12	
4	(l° l) ∪ (r° r) = 𝟙 Ex 5.12	

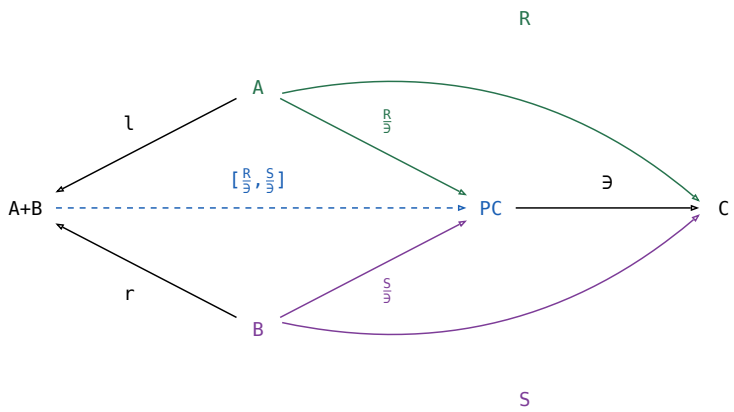
§5.3.1. [R,S] = [$\frac{R}{∋}$,$\frac{S}{∋}$] ∋ — a case of relations is the case of their transposes, then ∋

The universal-property row is not free: l,r were only ever asked to be a coproduct of **maps**. They stay one once every arrow is allowed because $\frac{\cdot}{∋}$ sends an arrow A→C to a map A→PC reversibly, so the map coproduct can be applied underneath it. For any T : A+B→C,

(5.3.1a)

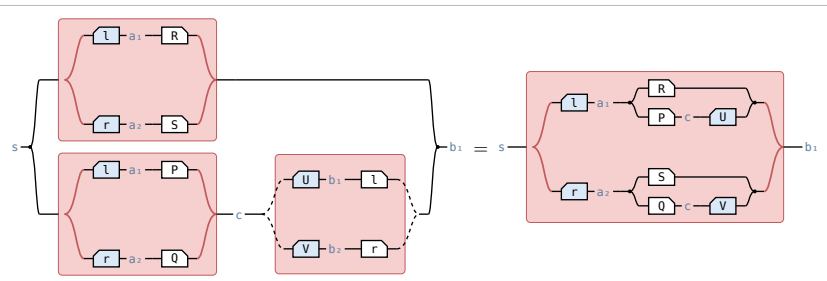


(5.3.1b)



Nothing here holds only up to ⊑: every triangle commutes on the nose, which is the difference from the fork above. The border spells `[R,S]=[`$\frac{R}{\ni}$`,` $\frac{S}{\ni}$`]∋`, and pushing ∋ into the union that `[·,·]` on maps already is turns that back into the definition, `(l°` $\frac{R}{\ni}$ `∪r°` $\frac{S}{\ni}$`)∋=l°R ∪ r°S`.

(5.3.1c)

	the law	**picture**
1	prove (5.11), and say why duality does not carry it over from the product law	
2	`(R + S) ∩ ([P,Q] [U,V]°) = (R ∩ (P U°)) + (S ∩ (Q V°))` `[P,Q][U,V]°` is a full 2×2 of composites; `R+S` is diagonal, so the meet cuts the two off-diagonal branches	

§5.4. The power relator P(R)

(5.4a)

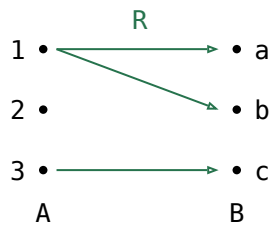
P(R) ≜ (∈ \ (R ∈)) ∩ ((∋ R) / ∋) — the power relator P(R) : PA↛PB, R lifted to sets.

xs P(R) ys⇔(∀a∈xs. ∃b∈ys. a R b)∧(∀b∈ys. ∃a∈xs. a R b)

(5.4b)

		in Rel	in words	im(xs)	partner
1	(∋R)/∋	∀y∈ys. ∃x∈xs. x R y	∀y. some xs R y	ys⊆im(xs)	every y has a partner
2	(∋/(∋R))°	∀y. (∃x∈xs. x R y)→y∈ys	every y with some xs R y is in ys	ys⊇im(xs)	ys leaves out no partner
3	E(R)	ys={y|∃x∈xs. x R y}	ys = every y with some xs R y	ys=im(xs)	every y has a partner, and none is left out
4	((∋R°)/∋)°	∀x∈xs. ∃y∈ys. x R y	∀x. x R some ys	—	every x has a partner
5	P(R)	P(R)(xs,ys) ⇔ (∀y. y ∈ ys ⇒ (∃x. x ∈ xs ∧ R(x,y))) ∧ (∀x. x ∈ xs ⇒ (∃y. y ∈ ys ∧ R(x,y)))	∀y. some xs R y and ∀x. x R some ys	—	every x and every y has a partner

(5.4c)



	xs	(∋R)/∋	(∋/(∋R))°	E(R)	((∋R°)/∋)°	P(R)
1	∅	∅	all 8 subsets of abc	∅	all 8 subsets of abc	∅
2	{1}	∅, a, b, ab	ab, abc	ab	a, b, ab, ac, bc, abc	a, b, ab
3	{2}	∅	all 8 subsets of abc	∅	none	none
4	{3}	∅, c	c, ac, bc, abc	c	c, ac, bc, abc	c
5	{1,2}	∅, a, b, ab	ab, abc	ab	none	none
6	{1,3}	all 8 subsets of abc	abc	abc	ac, bc, abc	ac, bc, abc
7	{2,3}	∅, c	c, ac, bc, abc	c	none	none
8	{1,2,3}	all 8 subsets of abc	abc	abc	none	none

(5.4d)

	law	the reading
1	X ⊑ P(R) ⇔ X ∋ ⊑ ∋ R ∧ X° ∋ ⊑ ∋ R°	One containment, and the same one at R° — which is the definition read off the two divisions. Hence P(R°)=P(R)°, and R⊑S⇒P(R)⊑P(S) .
2	P(𝟙) = 𝟙	The straightness axiom verbatim: extensionality **is** P's unit law.
3	Map(f) ⇒ P(f) = E(f)	In Rel, xs P(f) ys⇔ys={f(a)|a∈xs}. The half at f° says every a∈xs has its f(a) on ys; f has just the one image per a, so that already says ys contains everything xs reaches, which is the fraction's second half. For a map the two definitions coincide.
4	P(R S) = P(R) P(S)	⊒ is the division cancellation laws. ⊑ is the one law in this section that is not a calculation: it needs a tabulation of P(RS).

§5.4.1. Type relators

(5.4.1a)

T(R) = ⟮F(R,𝟙) α⟯ — the type functor T acts on an arrow R : A→B by rebuilding with α, applying R to the parameter.

(5.4.1b)

	name	law	what it says
1	the defining equation	T(R) = ⟮F(R,𝟙) α⟯	Rebuild the structure with α, applying R to the parameter on the way.
2	type relator	T(R)° = T(R°)	A datatype acts on relations, not only on maps — the map of the converse is the converse of the map.

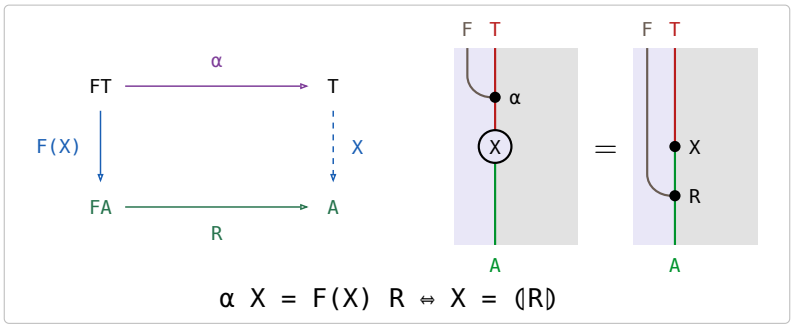
§5.5. Relational catamorphisms

(5.5a)

> The initial algebra `α : F(T)→T` among the maps is also initial in the allegory:

§5.5.1. The defining equation

(5.5.1a)



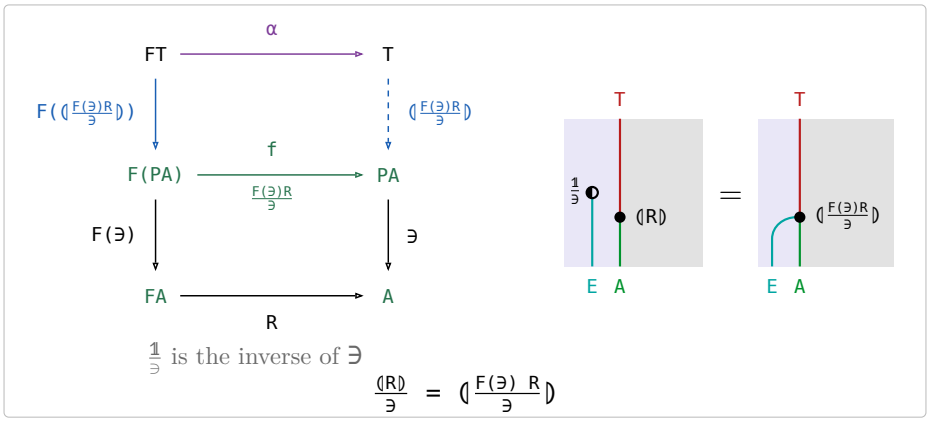
An F-algebra is a **weakened** natural transformation. A transformation `F⇒𝟙` on $\mathcal{A}$ would need a component `FX→X` at every object and a commuting square at every arrow, but F-Algebra only need it works on T and A.

(5.5.1b)

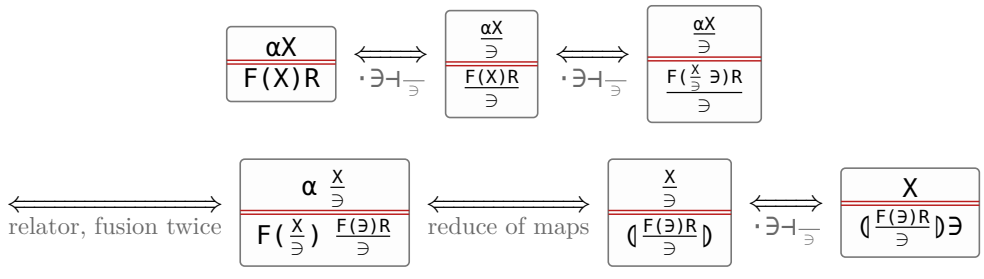
	`(|[c,f]|)=reduce(c,f)` catamorphism: arrows of the form `(|f|)` are called catamorphisms, and these two examples reveal the notion to be a familiar idea in abstract clothing	
1	**part**	**Nat**	**[A]**
2	datatype	`Nat::=zero|succ Nat`	`[A]::=nil|cons(A,[A])`
3	base functor `F`	`F(X)=1+X`	`F(X)=1+A×X`
4	initial algebra `α`	`α=[zero,succ]` `: 1+Nat→Nat`	`α=[nil,cons]` `: 1+A×[A]→[A]`
5	the fold, pointwise	`(|[c,f]|)(zero)=c` `(|[c,f]|)(succ(n))=f((|[c,f]|)(n))`	`(|[c,f]|)(nil)=c` `(|[c,f]|)(cons(a,x))=f(a,(|[c,f]|)(x))`

§5.5.2. (|R|) = (|$\frac{F(\ni)\ R}{\ni}$|) ∋ — folding with R is a fold of maps into sets, then ∋

(5.5.2a)



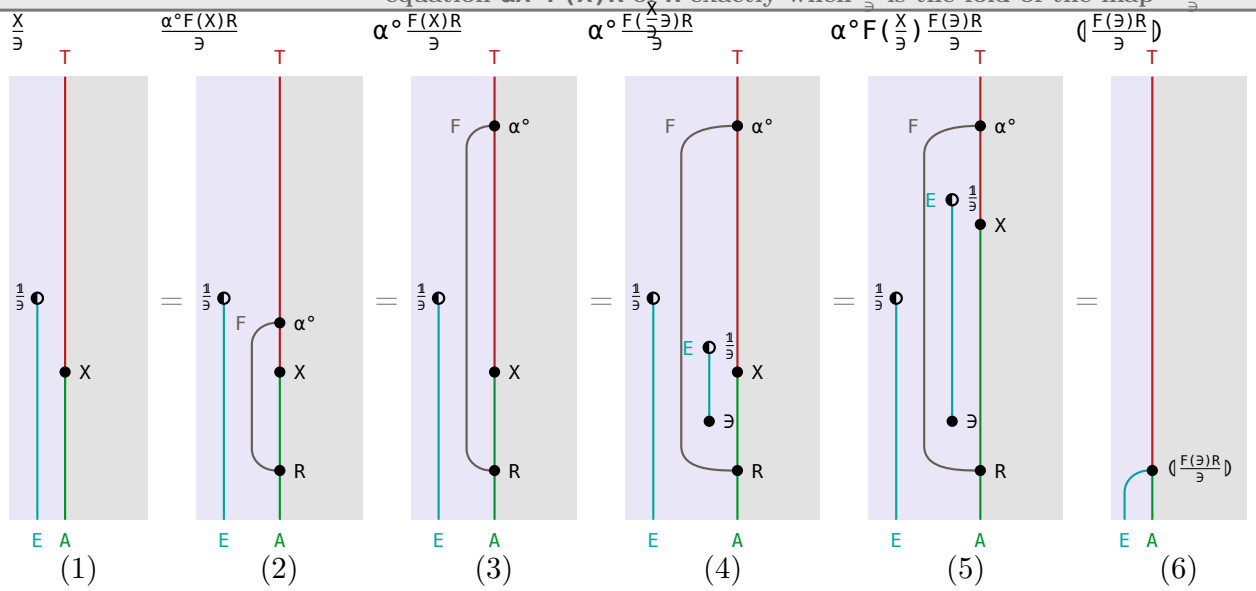
(5.12)



the last two rows at `X:=(|R|)`:

$$\alpha\ X = F(X)\ R \Leftrightarrow \frac{X}{\ni} = (\!|\frac{F(\ni)\ R}{\ni}|\!) \qquad (5.5.2b)$$

in a tabular allegory, with F a relator and $\alpha : F(T) \to T$ its initial algebra: a relation X out of T satisfies the fold equation $\alpha X = F(X) R$ of R exactly when $\frac{X}{\ni}$ is the fold of the map $\frac{F(\ni) R}{\ni}$



1	$\frac{X}{\ni}$	α iso; $\alpha X = F(X) R$
2	$= \frac{\alpha^\circ\ F(X)\ R}{\ni}$	α° is a map because α is an iso, so it leaves Λ: $\frac{\alpha^\circ F(X) R}{\ni} = \alpha^\circ \frac{F(X) R}{\ni}$ — `Map(f)` $\Rightarrow \frac{f\ R}{\ni} = f\ \frac{R}{\ni}$
3	$= \alpha^\circ\ \frac{F(X)\ R}{\ni}$	$X = \frac{X}{\ni}\ni$ — $\frac{R}{\ni}\ \ni = R$
4	$= \alpha^\circ\ \frac{F(\frac{X}{\ni})\ \ni)\ R}{\ni}$	$F(\frac{X}{\ni})\ni) = F(\frac{X}{\ni}) F(\ni)$ as F is a functor, and $F(\frac{X}{\ni})$ is a map (a relator sends maps to maps), so it leaves Λ — $\triangleq$ relator, `Map(f)` $\Rightarrow \frac{f\ R}{\ni} = f\ \frac{R}{\ni}$
5	$= \alpha^\circ\ F(\frac{X}{\ni})\ \frac{F(\ni)\ R}{\ni}$	fold uniqueness, $\frac{F(\ni) R}{\ni}$ being a map — `(h : f → g) ≜ f h = F(h) g , Map(f)` $\Rightarrow$ `α (⦇f⦈) = F(⦇f⦈) f`
6	$= (\!\|\frac{F(\ni)\ R}{\ni}\|\!)$	

§5.6. Combinatorial functions

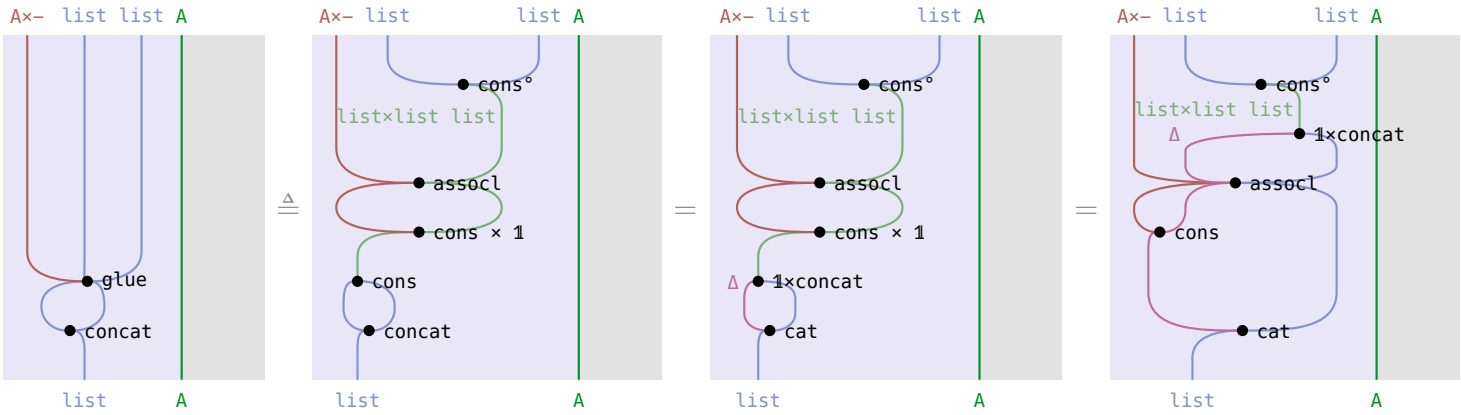
(5.6a)

	definition	type	note
1	`[A]::=nil|cons(A,[A])` `list(A) = [A]`	$\mathcal{A}\to\mathcal{A}$	The list type, under the short name it keeps.
2	`R′ = ⦇[nil,(R × 𝟙) cons]⦈`	`[A]→[B]`	The relator's action on `R : A→B`: one `R` per element, the shape untouched.
3	`subseq = ⦇[nil,cons ∪ π₂]⦈`	`[A]→[A]`	`xs subseq ys`: `ys` is `xs` with elements dropped — `cons` keeps the head, `π₂` drops it.
4	`prefix = ⦇[nil,cons ∪ ⊸ nil]⦈` `prefix = cat° π₁ =init*`	`[A]→[A]`	`ys` is an initial segment of `xs`; the first `nil` is where it stops early. `init≙snoc° π₁`.
5	`suffix = cat° π₂ =tail*`	`[A]→[A]`	The dual, `tail≙cons° π₂`; as a reduce it needs snoc-lists.
6	`partition = concat°`	`[A]→[[A]]`	This `cat` is restricted to `[A]⁺×[A]→[A]`, so `ys` is a list of non-empty segments of `xs`.
7	`concat = ⦇[nil,cat]⦈`	`[[A]]→[A]`	Joins the segments back up, which is why its converse splits a list.
8	`inits`	`[A]→[[A]]`	Implements $\frac{\texttt{prefix}}{\ni}$, listing the prefixes by increasing length.
9	`tails`	`[A]→[[A]]`	Implements $\frac{\texttt{suffix}}{\ni}$ by decreasing length — the opposite order.
10	`cpp`, `cpl`, `cpr`	`[A]×[B]→[A×B]` `[A]×B→[A×B]` `A×[B]→[A×B]`	Every pair `(a,b)` with `a` from the first list and `b` from the second; `cpl`, `cpr` keep one side a single element.
11	`cpp setify = (setify × setify)` $\frac{\ni\times\ni}{\ni}$ `cpl setify = (setify × 𝟙)` $\frac{\ni\times\mathbb{1}}{\ni}$ `cpr setify = (𝟙 × setify)` $\frac{\mathbb{1}\times\ni}{\ni}$	`[A]×[B]→P(A×B)`	Forgetting the order with `setify`, `cpp` is the transpose of `∋×∋`.
12	`cp(F)` $\triangleq \frac{\texttt{F}(\ni)}{\ni}$	`F(PA)→PFA`	`cp(F)`, the transpose of a relator's action on `∋`; `cpp` implements it at `F(A)=A×A`.
13	`cp(list) = ⦇`$\frac{\texttt{[nil,(∋ × ∋) cons]}}{\ni}$`⦈` $\frac{\texttt{[nil,(∋ × ∋) cons]}}{\ni}$ `= [nil` $\frac{\mathbb{1}}{\ni}$`,` $\frac{\ni\times\ni}{\ni}$ `E(cons)]`	`list(PA)→P(list(A))`	`cp(list)`: (5.12) `⦇R⦈ = ⦇`$\frac{\texttt{F}(\ni)\ \texttt{R}}{\ni}$`⦈ ∋` at `∋′=⦇[nil,(∋×𝟙)cons]⦈`, then the algebra expanded.
14	`cplist = ⦇[nil τ,cpp cons′]⦈`	`[[A]]→[[A]]`	`cp(list)` with every set a list: `cpp` in place of $\frac{\ni\times\ni}{\ni}$.

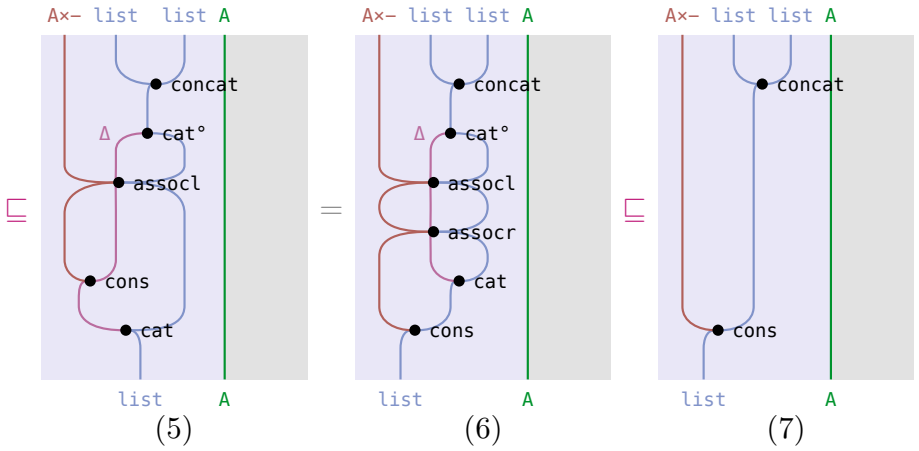
`glue concat ⊑ (𝟙 × concat) cons` (5.6b)

putting **a** on the front of the first segment and flattening (**`glue concat`**) gives at most flattening the segments and putting **a** on the front of the result (**`(𝟙×concat) cons`**)

the starting composite



1	`glue concat`	definition of glue
2	`≜ (𝟙 × cons°) assocl cons × 𝟙 cons concat`	since `concat=⦇[nil,cat]⦈`
3	`= (𝟙 × cons°) assocl cons × 𝟙 (𝟙 × concat) cat`	naturality of `assocl`
4	`= (𝟙 × (cons° (𝟙 × concat))) assocl cons × 𝟙 cat`	since `concat=⦇[nil,cat]⦈`



5	`⊑ (𝟙 × concat) (𝟙 × cat°) assocl cons × 𝟙 cat`	since `(cons×𝟙) cat=assocr (𝟙×cat) cons`
6	`= (𝟙 × concat) (𝟙 × cat°) assocl assocr (𝟙 × cat) cons`	since `assocl assocr=𝟙` and `cat° cat⊑𝟙`
7	`⊑ (𝟙 × concat) cons`	

§5.6.1. $\frac{\texttt{subseq}}{\ni}$ `= ⦇[nil` $\frac{\mathbb{1}}{\ni}$`,⟨`$\frac{\mathbb{1} \times \ni}{\ni}$ `E(cons),π₂⟩ cup]⦈` — the set of subsequences is a fold of maps into sets

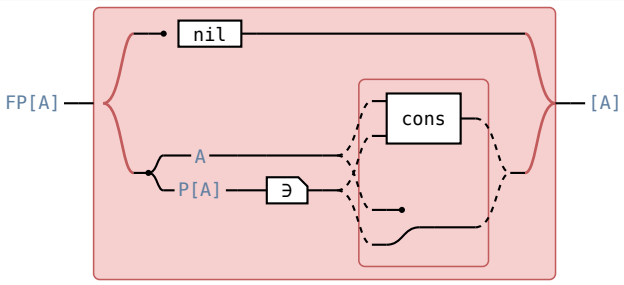
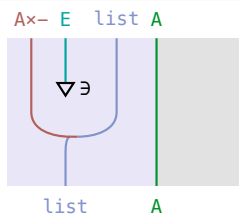
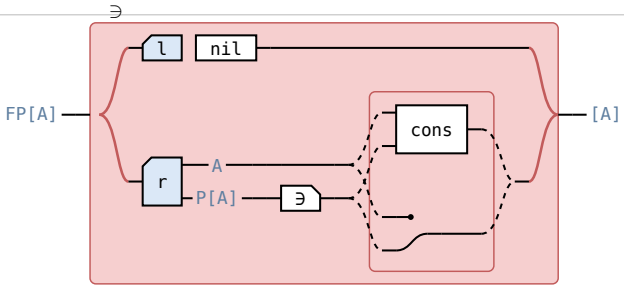
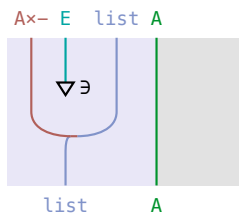
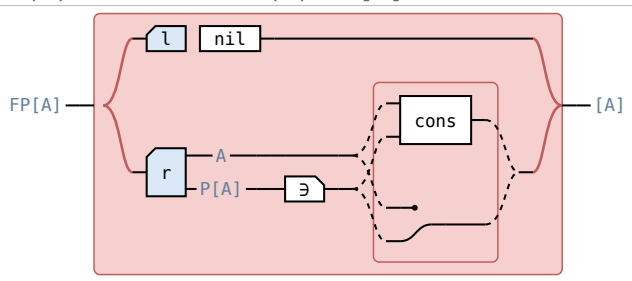
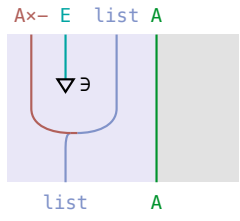
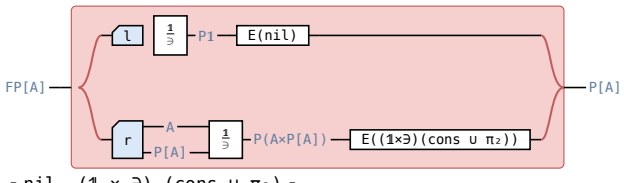
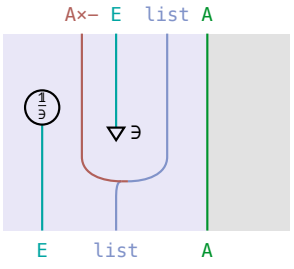
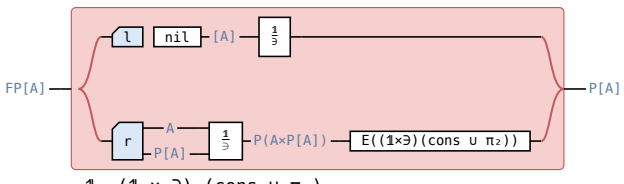
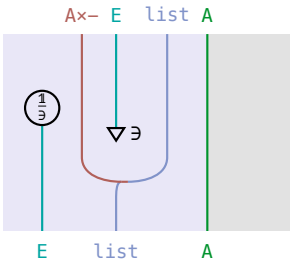
(5.6.1a)

`cup` $\triangleq \frac{(\pi_1\ \ni) \cup (\pi_2\ \ni)}{\ni}$ — `cup` unions two sets.

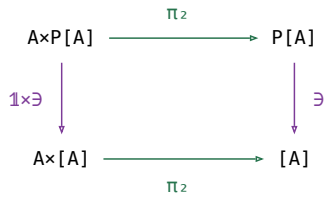
`cup`$\triangleq \frac{\pi_1 \ni \cup \pi_2 \ni}{\ni}$ `: PA×PA→PA`

$$\frac{\texttt{F(∋) [nil,cons ∪ π₂]}}{\texttt{∋}} \texttt{ = [nil } \frac{\texttt{𝟙}}{\texttt{∋}}\texttt{, } \frac{\texttt{(𝟙 × ∋) (cons ∪ π₂)}}{\texttt{∋}}\texttt{]} \qquad \textbf{(5.6.1b)}$$

the set of lists the algebra builds is, from nothing, just **nil**, and from a head and a set of tails, every tail in the set with the head put on or left off — (5.12) (|R|) = (|$\frac{\texttt{F(∋) R}}{\texttt{∋}}$|) ∋ at **subseq=(|[nil,cons ∪ π₂]|)**, **subseq = (|[nil,cons ∪ π₂]|)** .

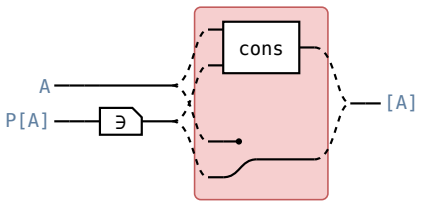
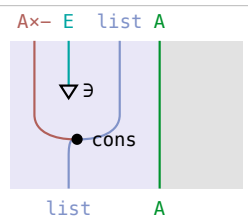
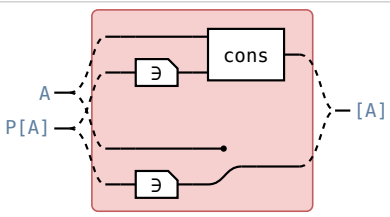
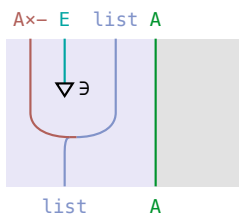
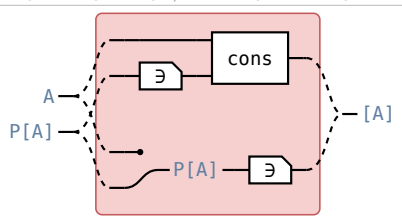
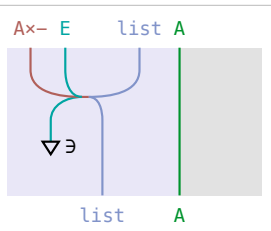
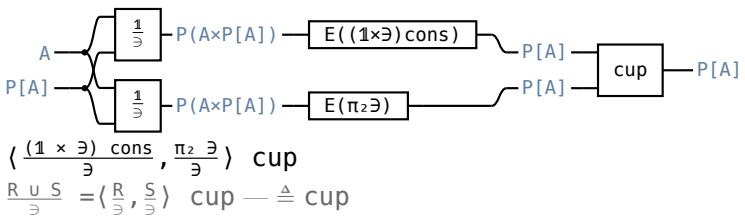
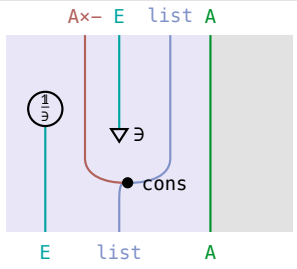
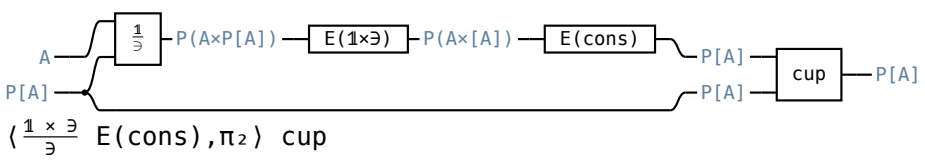
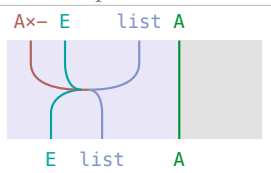
circuit — the fork is F([A])=1+A×[A]: nil above, the pair below	**Hinze–Marsden**
 $\frac{\texttt{F(∋) [nil,cons ∪ π₂]}}{\texttt{∋}}$	 the π₂ operand of cons ∪ π₂ under the 𝟙×∋ summand of F(∋), i.e. (𝟙×∋)π₂
=  $\frac{\texttt{(𝟙 + (𝟙 × ∋)) [nil,cons ∪ π₂]}}{\texttt{∋}}$ F(X)=1+A×X — list(A) = [A]	 the same operand under 𝟙+𝟙×∋, whose 𝟙×∋ summand it sits in
=  $\frac{\texttt{[nil,(𝟙 × ∋) (cons ∪ π₂)]}}{\texttt{∋}}$ R+S≜[Rl,Sr], l[R,S]=R, r[R,S]=S — [R,S] ≜ (l° R) ∪ (r° S) , l [R,S] = R , r [R,S] = S , l T = R; r T = S ⇒ T = [R,S]	 the π₂ operand of the second arm (𝟙×∋)(cons ∪ π₂)
=  $\left[\frac{\texttt{nil}}{\texttt{∋}}\texttt{, } \frac{\texttt{(𝟙 × ∋) (cons ∪ π₂)}}{\texttt{∋}}\right]$ [R,S] = [$\frac{\texttt{R}}{\texttt{∋}}$,$\frac{\texttt{S}}{\texttt{∋}}$] ∋ at T:=[nil,(𝟙×∋)(cons ∪ π₂)]	 the π₂ operand under its $\frac{\texttt{𝟙}}{\texttt{∋}}$, the arm (𝟙 × ∋) π₂ = π₂ ∋ rewrites
=  $\left[\texttt{nil } \frac{\texttt{𝟙}}{\texttt{∋}}\texttt{, } \frac{\texttt{(𝟙 × ∋) (cons ∪ π₂)}}{\texttt{∋}}\right]$ Map(f); 𝟙□ = ∋□ ⇒ $\frac{\texttt{f}}{\texttt{∋}}$ = f $\frac{\texttt{𝟙}}{\texttt{∋}}$, $\frac{\texttt{f}}{\texttt{∋}}$ =f $\frac{\texttt{𝟙}}{\texttt{∋}}$ for f a map, at f:=nil	 the same operand; the two rows differ only in the nil arm

(5.6.1c)

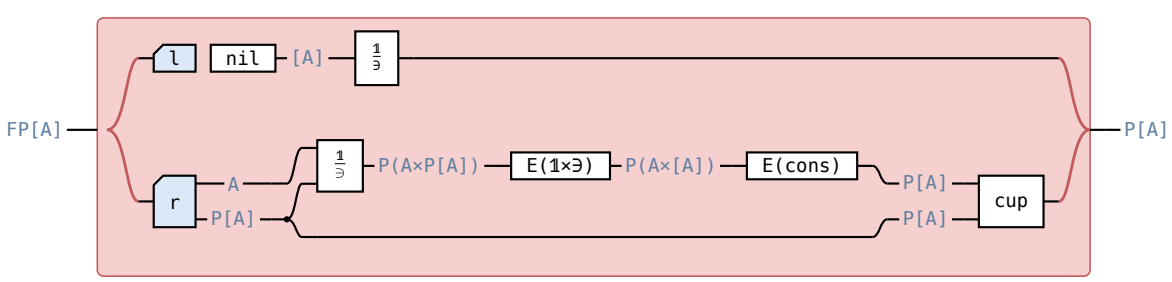


$\frac{(\mathbb{1} \times \ni)\ (\mathsf{cons} \cup \pi_2)}{\ni} = \langle \frac{\mathbb{1} \times \ni}{\ni}\ \mathsf{E(cons)}, \pi_2 \rangle\ \mathsf{cup}$ **(5.6.1d)**

power transpose of join: the power transpose of the join of two relations is $\langle \frac{R}{\ni}, \frac{S}{\ni} \rangle$ cup, where cup is the function that returns the union of two sets

circuit	Hinze–Marsden
 $\frac{(\mathbb{1} \times \ni)\ (\mathsf{cons} \cup \pi_2)}{\ni}$ $\frac{\mathsf{F}(\ni)\ [\mathsf{nil}, \mathsf{cons} \cup \pi_2]}{\ni} = [\mathsf{nil}\ \frac{\mathbb{1}}{\ni}, \frac{(\mathbb{1} \times \ni)\ (\mathsf{cons} \cup \pi_2)}{\ni}]$'s second branch	 the cons operand of $\mathsf{cons} \cup \pi_2$
=  $\frac{((\mathbb{1} \times \ni)\ \mathsf{cons}) \cup ((\mathbb{1} \times \ni)\ \pi_2)}{\ni}$ $\mathsf{T}(X_1 \cup X_2) = \mathsf{T}X_1 \cup \mathsf{T}X_2$ — $(S \cup T)\ R = (S\ R) \cup (T\ R)$, $R\ (S \cup T) = (R\ S) \cup (R\ T)$, $R \cap (S \cup T) = R \cap S \cup R \cap T$, $(R \cup S)^\circ = S^\circ \cup R^\circ$	 the π_2 operand of $(\mathbb{1} \times \ni)\mathsf{cons} \cup (\mathbb{1} \times \ni)\pi_2$
=  $\frac{((\mathbb{1} \times \ni)\ \mathsf{cons}) \cup (\pi_2\ \ni)}{\ni}$ $(\mathbb{1} \times \ni)\pi_2 = \pi_2 \ni$ — $(R \times S)\ \pi_1 \sqsubseteq \pi_1\ R \wedge (R \times S)\ \pi_2 \sqsubseteq \pi_2\ S$ at π_2, an equality because $\mathbb{1}$ is entire	 the π_2 operand of $(\mathbb{1} \times \ni)\mathsf{cons} \cup \pi_2 \ni$
=  $\langle \frac{(\mathbb{1} \times \ni)\ \mathsf{cons}}{\ni}, \frac{\pi_2\ \ni}{\ni} \rangle$ cup $\frac{R \cup S}{\ni} = \langle \frac{R}{\ni}, \frac{S}{\ni} \rangle$ cup — $\triangleq$ cup	 the cons operand under its $\frac{\mathbb{1}}{\ni}$
=  $\langle \frac{\mathbb{1} \times \ni}{\ni}\ \mathsf{E(cons)}, \pi_2 \rangle$ cup $\frac{S}{\ni}\ \mathsf{E}(R) = \frac{S\ R}{\ni}$, absorption $\frac{S}{\ni}\ \mathsf{E}(R) = \frac{SR}{\ni}$ at $S := \mathbb{1} \times \ni$, $R := \mathsf{cons}$; fusion and $\frac{\ni}{\ni} = \mathbb{1}$ on the π_2 operand	 the π_2 operand, bare π_2

(5.6.1e)



$[\frac{\mathsf{nil}}{\ni}, \langle \frac{\mathbb{1} \times \ni}{\ni}\ \mathsf{E(cons)}, \pi_2 \rangle\ \mathsf{cup}]$

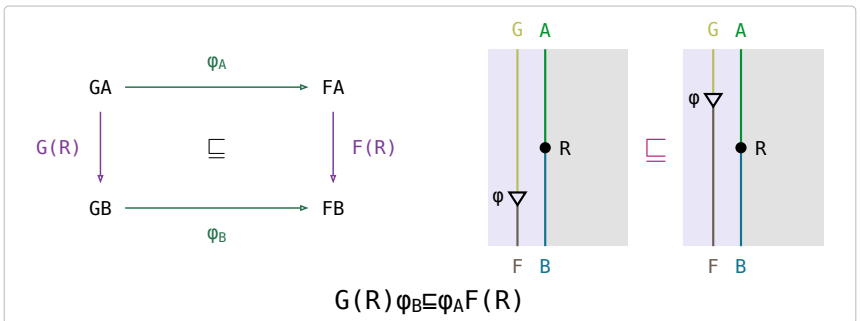
which writes Pcons; cons is a map, and there $\mathsf{P(cons)} = \mathsf{E(cons)}$ — $\mathsf{Map}(f)$ $\Rightarrow \mathsf{P}(f) = \mathsf{E}(f)$.

§5.7. Lax natural transformations (LaT)

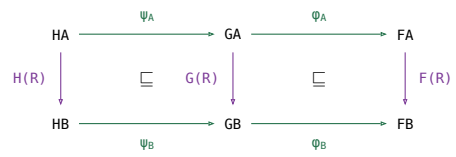
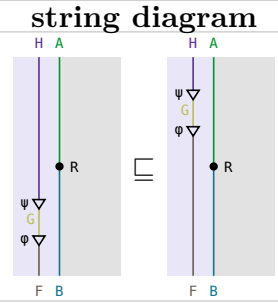
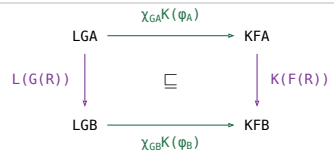
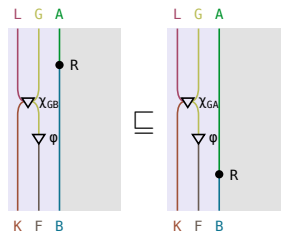
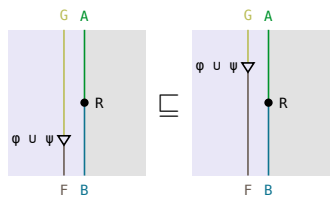
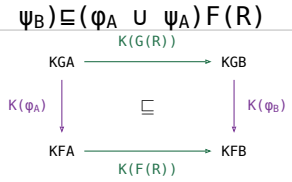
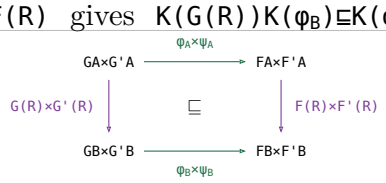
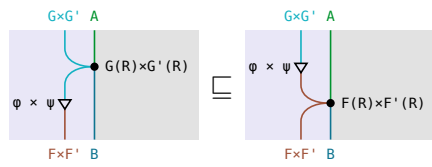
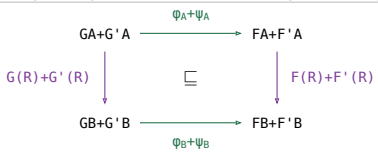
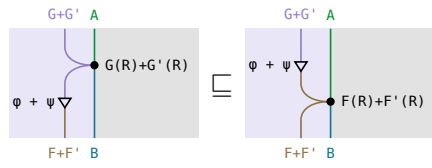
(5.7a)

φ : G⇒F is **lax at** R : A↛B when G(R)φ$_B$⊑φ$_A$F(R); it is a **lax natural transformation** (LaT) when lax at every R.

(5.7b)



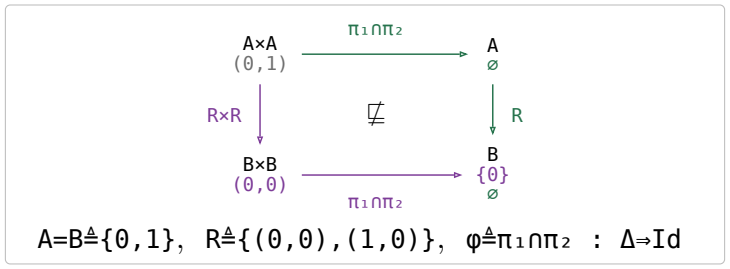
(5.7c)

	closed under	commutative diagram	string diagram
1	composition ψφ	 H(R)ψB⊑ψAG(R) and G(R)φB⊑φAF(R) give H(R)(ψBφB)⊑(ψAφA)F(R)	
2	horizontal composition χ∘φ	 φ : G⇒F with G,F : 𝒞→𝒟 and χ : L⇒K with L,K : 𝒟→ℰ give χ∘φ : L∘G⇒K∘F, the family A ↦ χGAK(φA) L(G(R))χGB⊑χGAK(G(R)) is χ lax at G(R); then K(G(R)φB)⊑K(φAF(R)) is K applied to φ's own inequation; χ:=𝟙K gives K(φ) : K∘G⇒K∘F, K composed on the outside; φ:=𝟙G gives χG : L∘G⇒K∘G, G composed on the inside two candidates, χGAK(φA) and L(φA)χFA, ordered by ⊑; this row is the first	
3	union φ ∪ ψ	 G(R)φB⊑φAF(R) and G(R)ψB⊑ψAF(R) give G(R)(φB ∪ ψB)⊑(φA ∪ ψA)F(R)	
4	a relator K K(φ)	 G(R)φB⊑φAF(R) gives K(G(R))K(φB)⊑K(φA)K(F(R))	
5	product φ×ψ	 φ : G⇒F and ψ : G'⇒F' give φ×ψ : G×G'⇒F×F' (R×S)(U×V)=(RU)×(SV) and monotonicity in both slots — the row above's K applied to the inequation, run on a bifunctor; the fork is the derived case ⟨φ,ψ⟩=◁(φ×ψ), and its ONE cost is the lax copy law R◁⊑◁(R×R) — R ◁ ≤ ◁ (R ⊗ R) , R ⊸ ≤ ⊸	
6	coproduct φ+ψ	 φ : G⇒F and ψ : G'⇒F' give φ+ψ : G+G'⇒F+F' (R+S)(U+V)=(RU)+(SV) and monotonicity in both slots; the co-fork is the derived case [φ,ψ]=(φ+ψ)▿, and ▿ costs nothing	
7	meet — **fails** φ∩ψ	the step it would need is (φAF(R))∩(ψAF(R))⊑(φ∩ψ)AF(R), the wrong direction of R (S ∩ T) ⊑ (R S) ∩ (R T) a counterexample: aa = (0,1); bb = (0,0); (R × R)(aa,bb); b₁ = {0}; b₁ = E(π₁ ∩ π₂){bb}; a₁ = ∅; a₁ = E(π₁ ∩ π₂){aa}; b₂ = ∅; b₂ = E(R)(a₁) ⇒ ¬((R × R) (π₁ ∩ π₂) ⊑ (π₁ ∩ π₂) R)	
8	Each row is the general slide RAX⊑X'RB and RBY⊑Y'RC give RA(XY)⊑(X'Y')RC at RA,RB:=G(R),F(R) and X,X':=φB,φA, quantified over R; at X'=X with RA,RB endorelations it is the monotonicity reading instead — the ∀R sits outside the law, and is the only difference		

§5.7.1. meet is not closed in LaT

(5.7.1a)

counterexample — φ,ψ : G⇒F lax natural does NOT give φ∩ψ lax natural



A=B≜{0,1}, R≜{(0,0),(1,0)}, φ≜π₁∩π₂ : Δ⇒Id

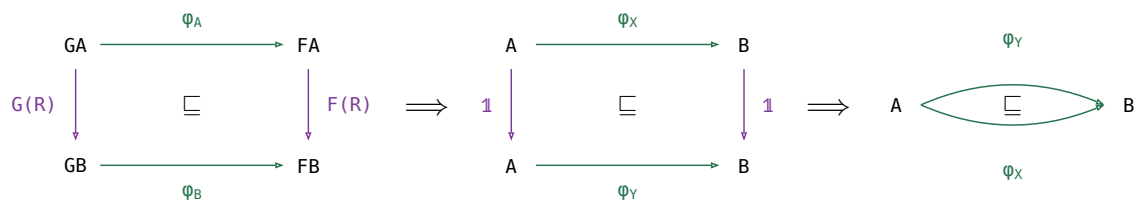
§5.7.2. Two stacked towers

(5.7.2a)

		lower — inside ONE allegory	upper — between allegories
1	0-cell	an object A	an allegory $\mathcal{C}$ — **a whole lower tower**
2	1-cell	a relation R : A↛B	a relator F : $\mathcal{C}$↛$\mathcal{D}$
3	2-cell	R⊑S	a LaT φ : G⇒F ≙ lax
4	order	R⊑S	φ⊑ψ componentwise
5	union	R ∪ S	φ ∪ ψ, **survives** lax(φ); lax(ψ) ⇒ G(R) (φB ∪ ψB) ⊑ (φA ∪ ψA) F(R)
6	product	R×S ≙ ×	φ×ψ, **survives**; a TENSOR, not a categorical product; SYMMETRIC ⟨R,S⟩ π₁ = dom(S) R , ⟨R,S⟩ π₂ = dom(R) S lax(φ); lax(ψ) ⇒ (G(R) × G'(R)) (φB × ψB) ⊑ (φA × ψA) (F(R) × F'(R))
7	coproduct	R+S	φ+ψ, **survives**; a BIPRODUCT lax(φ); lax(ψ) ⇒ (G(R) + G'(R)) (φB + ψB) ⊑ (φA + ψA) (F(R) + F'(R))
8	meet	R∩S	φ∩ψ componentwise, **fails** aa = (0,1); bb = (0,0); (R × R)(aa,bb); b₁ = {0}; b₁ = E(π₁ ∩ π₂){bb}; a₁ = ∅; a₁ = E(π₁ ∩ π₂){aa}; b₂ = ∅; b₂ = E(R)(a₁) ⇒ ¬((R × R) (π₁ ∩ π₂) ⊑ (π₁ ∩ π₂) R)
9	converse	R°	φ°, **fails**, oplax: φA°G(R)⊑F(R)φB°
10	zero object	z with 𝟙 z=𝟘	the constant relator at such a z, initial and terminal

§5.7.3. R⊑S is a special case of LaT

(5.7.3a)



G≜const A, F≜const B — the relators X↦A and X↦B, each sending every arrow to 𝟙 ≙ lax

with $\mathcal{C}$ two objects and one non-identity arrow X→Y, a LaT const A⇒const B is the pair φX,φY : A↛B with φY⊑φX — the datum R⊑S

not every LaT is constant: ∋ : P⇒Id X ⊑ P(R) ⇔ X ∋ ⊑ ∋ R ∧ X° ∋ ⊑ ∋ R° π₂ : (A×−)⇒Id (𝟙 × ∋) π₂ = π₂ ∋ ◁ : Id⇒Δ R ◁ ≤ ◁ (R ⊗ R) , R ⊸ ≤ ⊸